Deploy, Prompt, And Refine

Ship the Base44 app to GitHub Pages, then deepen the program through contextual engineering

Where We Are Today

The last five class days have been one connected build story.

Teams started with real city budgets, found department priorities, researched programs, mapped Chamber partners, designed win-win-win program concepts, created software ideas, moved into Stitch, and then used Base44 to turn those ideas into working app prototypes.

Some parts moved fast. Some parts may have felt semi-blind in the moment. That is because we were walking through a full pipeline before stopping to name every piece of it.

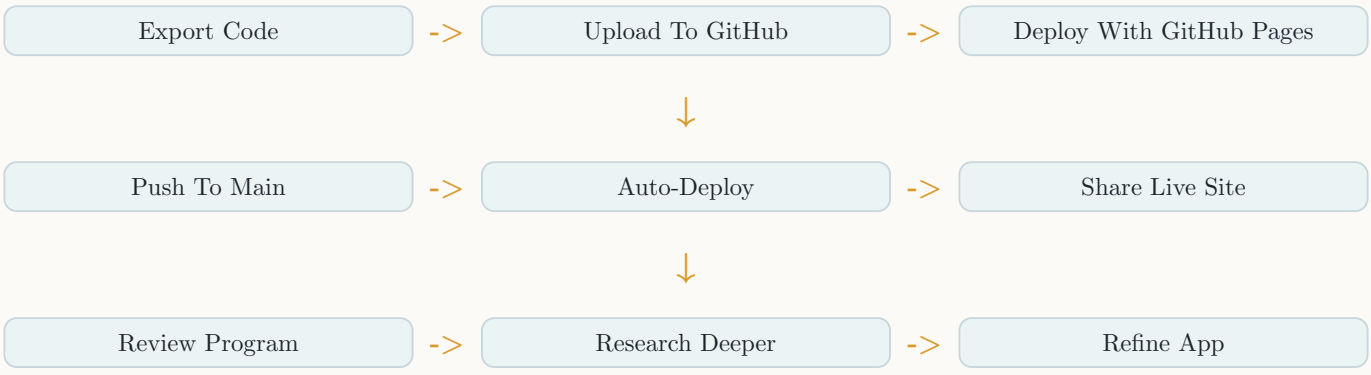
Today we go deeper.

> **Today we name the full story, ship the app, and refine the program behind the software.**

The first part of class is technical. We will export the Base44 code, upload it to GitHub, and deploy the frontend with GitHub Pages.

The second part is product work. We will review the program each team created, improve the activities, strengthen the flow, connect the app more tightly to the speakers and concepts being taught, and use research to make the experience more specific.

Today's Flow



The Tutorial Narrative

Today’s tutorial is not only about GitHub.

The tutorial is the full story of how a local opportunity becomes a live software offer:

From Opportunity To Software Offer

Here is the narrative:

1. A city budget shows what problems the city already spends money on.
2. Departments show who owns those problems and what programs already exist.
3. Program research shows what has been tried, what gaps still exist, and what pain points matter.
4. Chamber directories show businesses, nonprofits, and service providers that can help deliver a solution.
5. The team becomes the broker by designing a win-win-win program around the department, partners, and participants.
6. The software tool makes the program easier to run, easier to track, and easier to pitch.
7. Deployment turns the prototype into a live website that stakeholders can actually open.
8. Contextual engineering helps the team refine the app until it feels valid, specific, and attractive.

What students have really been building

You are not just building an app. You are building evidence that your team can understand local demand, organize collaborators, design a program, and provide the software system that helps the program run.

How We Got Here

Some of the work across the last five days may have felt fast or semi-blind.

That is normal. The class was moving through a full opportunity-to-software pipeline:

The Full Opportunity Pipeline

The prompts were not random. They were guiding teams through the way real consultants, product strategists, and software builders find demand, map stakeholders, create a program offer, and then build a tool that makes the offer more valuable.

Prompt Stage	What It Was Really Doing
Budget upload	Turning the city fiscal year plan into a demand map: departments, funding, priorities, and issues the city already cares about.
Department table	Finding department directors, staff, budgets, programs, plans, and pain points so the team could speak to real decision-makers.
Five-year program research	Studying what the department has already tried so the new idea does not feel disconnected from local reality.
Chamber partner search	Finding private-sector or nonprofit supply that could help deliver the program while creating partner ROI.
Program mechanics	Turning an idea into a structured program with roles, flows, lessons, activities, deliverables, and operating logic.
Software tool table	Identifying where the student company can enter as the technology partner and create value.
Three-day transformation blueprint	Defining the audience, promise, lessons, activities, data, partner roles, metrics, and final showcase.
Product architecture	Translating the program into app users, dashboards, forms, data flows, tracking, and impact reports.
Stitch design	Creating the visual portal structure so the app had a clear user experience before coding.
Base44 build	Turning the blueprint and design into a working software prototype with auth, dashboards, activities, and sample data.

> **Today we make the software feel less like a demo and more like a serious offer.**

That means each team should refine for two audiences at the same time:

- Demand managers: department leaders, program managers, city staff, school staff, or community leaders who own the problem.
- Collaborators: Chamber businesses, nonprofits, service providers, sponsors, mentors, venues, and other partners who can help deliver the program.

Your app should make both groups think:

The credibility test

This team understands the problem, knows the stakeholders, has a practical program, can track outcomes, and built a tool that would make the program easier to run.

Why We Are Deploying

Base44 helped teams build quickly. Now the app needs a real public home.

When we deploy to GitHub Pages, each team gets a live website they can open, test, share, and keep improving. The goal is not just to have code sitting in a ZIP file. The goal is to create a workflow where the app can keep changing.

The deployment goal

When Base44 pushes new code to the main branch, GitHub should automatically rebuild the frontend and publish the newest version of the app.

That means students get a real software delivery loop:

- Build in Base44
- Export or push code
- Store code in GitHub
- Deploy through GitHub Actions
- Test the live website
- Refine the app
- Push again
- See the website update

The Backend Question

Most Base44 apps are not just static pages. They usually point to a backend, database, authentication system, API, or Base44-hosted service.

When we move the frontend to GitHub Pages, we need to make sure the deployed site still knows where that backend is.

> **The frontend can live on GitHub Pages, but it still needs to talk to the same backend it was using before.**

Before deploying, each team should identify:

Question	What To Check
Where is the backend?	Look for API URLs, environment variables, auth settings, database URLs, or Base44 service configuration.
Is the app fully static?	If it only uses frontend code and external services, GitHub Pages may work directly.
Does it need secrets?	Secrets should not be placed directly in public frontend code. Use public-safe keys only.
Does the backend allow this domain?	Some auth or API tools require adding the GitHub Pages URL as an allowed origin or redirect URL.
Does routing work on refresh?	Single-page apps may need special handling so deep links do not 404 on GitHub Pages.

GitHub Pages Deployment Tutorial

We will walk through this together in class.

The exact commands may change depending on the Base44 export, but the idea is the same for most frontend apps.

Step 1: Export Code From Base44

In Base44, export the project code as a ZIP file.

Download the ZIP and unzip it on your computer.

Inside the folder, look for the frontend project structure. Most generated frontends will have files like:

- `package.json`
- `src/`
- `public/`
- `index.html` or a framework entry file
- `vite.config.js`, `next.config.js`, or another build config

What package.json tells us

The `package.json` file usually tells us how to install, run, and build the app. Look for scripts like `dev`, `build`, `preview`, or `start`.

Step 2: Create A GitHub Repository

Each team will create a GitHub repository for the app.

Use a clear repository name connected to the program, such as:

- `youth-career-launchpad`
- `healthy-neighborhoods-portal`
- `small-business-accelerator-app`
- `arts-access-program`

Keep the repository public if you want to use the standard free GitHub Pages workflow.

Step 3: Upload The Code

The first upload can happen through the GitHub website or through Git commands.

The important part is that the project files end up on the `main` branch.

- ☐ The repository has the unzipped Base44 project files
- ☐ `package.json` is at the correct project root
- ☐ The code is pushed to the `main` branch
- ☐ The team can see the files on GitHub
- ☐ The app still has its backend/API configuration
- ☐ No private secrets were pasted into public code

Step 4: Add The GitHub Actions Workflow

GitHub Actions is the CI/CD system.

CI/CD means:

- Continuous Integration: GitHub checks and builds the code when it changes.
- Continuous Deployment: GitHub publishes the new site after the build passes.

For most Vite-style frontend exports, the workflow will look similar to this:

GitHub Pages Workflow

Create a file named `.github/workflows/pages.yml`.

Use this structure as the starting point:

```yaml
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: ./dist

  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    needs: build
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

If the build output folder is not `dist`, we will adjust the workflow.

Framework / Build	Common Output Folder
Vite	`dist`
Create React App	`build`
Next static export	`out`
Plain HTML/CSS/JS	Project root or a configured folder

Step 5: Turn On GitHub Pages

In the repository settings:

1. Go to `Settings`
2. Go to `Pages`
3. Choose `GitHub Actions` as the source
4. Save the setting
5. Push to `main` and watch the Actions tab

When the workflow finishes, GitHub will give the team a live website URL.

Step 6: Test The Live Site

Testing the deployed site matters more than seeing a green checkmark.

- ☐ The landing page loads
- ☐ Images, styles, and icons appear
- ☐ Navigation works
- ☐ Refresh does not break important pages
- ☐ Login or sign-up still works if the app has auth
- ☐ Forms still submit to the correct backend
- ☐ Dashboard data still loads
- ☐ The app works on a phone-sized screen

What Happens After Deployment

Once the live site works, the team has a real product loop.

From this point forward, Base44 can make changes, push them to the `main` branch, and GitHub Actions can publish the updated site.

> **The live website becomes the testing ground for the program.**

Every team should save:

- GitHub repository link
- Live GitHub Pages link
- Base44 app link
- Current backend/API notes
- GitHub Actions status
- Known deployment issues
- Next refinement prompts

Prompt Engineering Gets Deeper Today

Across the first part of the program, we used prompt engineering to move through research, strategy, design, and app building.

Today we practice contextual engineering.

Prompt engineering is asking AI for the right output.

Contextual engineering is building the full context around the AI so it can help improve the whole system.

Prompt Engineering	Contextual Engineering
Ask for a page	Give the program goal, audience, research, data, partners, and desired transformation.
Ask for an activity	Explain what the speaker taught, what students should practice, and what evidence the app should collect.
Ask for a dashboard	Explain who uses it, what decisions they need to make, and what success metrics matter.
Ask for improvements	Compare the app against the department priority, award-winning examples, partner ROI, and user journey.

Today's deeper skill

We are not just prompting for screens. We are giving AI the research, goals, constraints, examples, and feedback it needs to help us refine the program.

What To Fine Tune Today

The app already has a first version. Today, each team should decide which specific part of the flow needs the most improvement.

Do not try to improve everything at once. Pick the part that will make the program feel more valid, more useful, or more attractive to a real stakeholder.

Flow Area	What To Improve
Demand story	Make the landing page connect directly to the city department, budget priority, local pain point, and audience need.
Program promise	Make the one-sentence promise clear enough that a department lead or partner understands the value in 10 seconds.
Partner ROI	Show why each collaborator benefits: visibility, referrals, customers, workforce pipeline, community impact, data, or relationship building.
Participant journey	Make the three-day flow feel like a transformation instead of three disconnected tasks.
Interactive activities	Turn passive reading into forms, choices, challenges, reflections, submissions, resource matches, or final artifacts.
Data collection	Collect evidence that proves progress, participation, learning, behavior change, partner engagement, or program outcomes.
Organizer dashboard	Give demand managers a clear view of signups, completion, blockers, submissions, partner usage, and impact metrics.
Final showcase	Make the final deliverable something the participant can show and the department or partner can understand quickly.

Flow Selection Prompt

“Review our current app and program.

Context:

- City: [Grand Rapids or Detroit]
- Department: [insert department]
- Department priority or program: [insert priority]
- Target audience: [insert audience]
- Main partners: [insert partners]
- Our company role: We are the technology partner helping run, track, and prove the program.

Identify the three weakest parts of the current flow.

Rank them by which improvement would create the most validity and attraction for:

1. The department or demand manager
2. The Chamber partners or collaborators
3. The participants
4. Our student company as the software partner

For each weak spot, recommend a specific Base44 prompt to improve the app.“

Refining The Application

After deployment, teams will review the program they created and improve the app around it.

The app should not feel like a random set of pages. It should flow with the concepts being taught by the speakers and the transformation the program promises.

Ask:

- What does the participant learn first?
- What should they practice next?
- What should they create?
- What should they submit?
- What should the app track?
- What should the team, department, or partner be able to see?
- What evidence proves the program worked?

Refinement Loop

Make Activities More Interactive

Today, teams should improve the app by adding stronger activities.

An activity is not just text on a page. A strong activity asks the user to do something, submit something, reflect on something, or make a decision.

Weak Activity	Stronger Interactive Activity
Read about the program	Answer an intake question that personalizes the next step.
Watch a lesson	Complete a short challenge connected to the lesson.
Write a reflection	Use guided prompts with saved progress and feedback.
Look at resources	Choose a partner resource and explain how it helps.
Finish the program	Submit a final artifact, plan, pitch, or showcase item.

Activity Refinement Prompt

"Review our current app activities and make them more interactive.

Context:
- Program name: [insert program]
- Target audience: [insert audience]
- Main transformation: [starting point -> ending point]
- Concepts taught by the speakers: [insert concepts]
- Partners/resources available: [insert partners]

Improve the app so each activity includes:
- A clear purpose
- A user action
- A saved submission
- A reflection question
- A progress state
- A connection to the program transformation
- Data that helps organizers understand impact"

Refinement Prompt Stack

Use these prompts after the app is deployed and the team has chosen which part of the flow to improve first.

Demand Manager Validity Prompt

“Improve our app so it feels credible to a department lead, city staff member, program manager, or other demand manager.

Use this context:

- Department: [insert department]
- Budget priority or pain point: [insert priority]
- Existing programs we researched: [insert examples]
- Target audience: [insert audience]
- Program promise: [insert promise]

Improve the landing page, program overview, organizer dashboard, and impact metrics so a decision-maker can quickly see:

- Why this program matches their real priority
- Who it serves
- What happens during the three days
- What partners are involved
- What data is collected
- What outcomes can be reported
- Why our software makes the program easier to run“

Collaborator Attraction Prompt

“Improve the app so collaborators and Chamber partners can clearly see why they should join this program.

Partners: [insert partner list]

For each partner, improve the app so it explains:

- What the partner contributes
- Which activity they support
- What audience they reach
- What business, community, recruiting, visibility, or impact value they receive
- What data proves their role mattered
- What follow-up opportunity the program creates

Add partner cards, ROI sections, partner dashboard details, and any missing resource links.“

Program Flow Tightening Prompt

“Tighten the three-day program flow inside the app.

Make the program feel like a clear transformation:

- Day 1 should diagnose the starting point and introduce the core concept.
- Day 2 should help participants practice with partners, resources, or service providers.
- Day 3 should help participants create a final artifact, plan, pitch, showcase, or submission.

For each day, improve:

- Lesson title
- Activity instructions
- User action
- Submission form
- Reflection prompt
- Progress state
- Partner connection
- Data collected
- How it prepares the participant for the final showcase“

Software Offer Prompt

“Improve the app so our student company looks like a serious software partner.

The app should show that we can:

- Help participants complete the program
- Help organizers manage the program
- Help partners see ROI
- Help the department understand impact
- Collect useful data
- Generate a final report or showcase

Add or improve features that make our software offer more valuable, pitchable, and practical.“

Research Before You Refine

Some teams may go back and research programs that have already won awards, earned recognition, or produced strong results in their department area.

That research can make the app sharper.

For example, if your program is connected to parks, public health, youth employment, small business, housing, public safety, sustainability, or arts and culture, look for:

- Award-winning city programs
- Department reports
- Case studies
- Program dashboards
- Community partner examples
- Metrics used to prove success
- Activities that participants actually complete

Research rule

Do not copy another program. Study what works, then adapt the pattern to your audience, city, partners, and 3-day transformation.

Award-Winning Program Research Prompt

“Help us research strong examples for our program area.

Program area: [parks / health / youth employment / small business / housing / public safety / sustainability / arts / other] City or department priority: [insert priority] Target audience: [insert audience]

Find patterns from award-winning or high-performing programs:

- What made the program effective?
- What activities did participants complete?
- What partners were involved?
- What data did the program track?
- What outcomes did it report?
- What could we adapt into our 3-day program and app?

Turn the findings into 10 specific improvements for our Base44 app.“

What I Will Be Doing In Class

After the deployment tutorial, I will come around to each group.

We will look at your live app, your program concept, your activities, your partner model, and your tracking. The goal is to help each group make the tool more refined for what they are trying to build.

Be ready to show:

- ☐ Your GitHub repository
- ☐ Your deployed GitHub Pages site
- ☐ Your Base44 project
- ☐ Your program blueprint
- ☐ Your target audience
- ☐ Your three-day activity flow
- ☐ Your partner list
- ☐ What data your app collects
- ☐ What feels weak or unfinished
- ☐ What you want the app to become next

Today's Deliverable

By the end of class, each team should have:

- ☐ Base44 code exported or connected to GitHub
- ☐ A GitHub repository on the main branch
- ☐ A GitHub Actions deployment workflow
- ☐ A live GitHub Pages website
- ☐ Backend/API configuration checked
- ☐ At least one successful push-to-main deployment
- ☐ A refined program flow
- ☐ More interactive activities
- ☐ A clearer tracking or impact plan
- ☐ Research notes that support the next iteration
- ☐ A list of the next Base44 prompts to run
- ☐ A stronger app that matches the program vision

The Big Idea

Today connects two worlds.

First, we learn how builders ship software so other people can use it.

Then we learn how builders refine software so it becomes more useful, more specific, and more connected to a real human outcome.

> **Deployment makes the app real. Contextual engineering makes the app worth using.**